

Plan Review and Implementation Recommendations

Executive Assessment

The documentation set (9 world docs + 14 architecture docs) describes an ambitious multi-agent economic simulation. The design is comprehensive, internally consistent on the main axes, and grounded in real biotech/pharma economics. It is also too large to build all at once. This review identifies what to build first, what to defer, what to simplify, and what needs more precision before coding begins.

1. What Works Well

Grounded world context. The SRT longevity drug setting is intuitive. An LLM reading these docs knows what a firm IS, what it makes, who buys it, and why R&D matters. This is the hardest part to get right and it is done.

Configurable complexity. The toggle system (doc 14) is the right design. A minimal run with 3 firms and 20 quarters should work before the full 5-firm, 80-quarter, all-features-on simulation is attempted.

Agent-as-application. The decision to give each agent its own database, tools, and multi-step reasoning is sound. It solves the "LLM produces nonsense numbers" problem by grounding decisions in statistical analysis before the LLM reasons.

Separation of truth. The orchestrator-as-sole-authority design is correct. Agents propose, the orchestrator disposes. This prevents accounting errors from propagating.

Single source for parameters. World doc 09 (Parameters and Calibration) as the canonical location for every number is essential. This was a gap and it is now closed.

2. Recommended Simplifications (for v1)

2a. Agent Count: Start with 8, Not 11

The 4-financial-agent split (Equity Market, Investment Bank, Commercial Bank, Credit Fund) is realistic but creates coordination complexity that does not pay off until the core simulation works.

Recommendation for v1:

- Merge Investment Bank into Equity Market. The IBank's advisory role

(structuring IPOs, M&A advisory) is small enough to be a sub-task of the Equity Market agent. When a firm requests equity, the Equity Market both structures and prices it. Remove the IBank as a separate agent.

- Keep Commercial Bank and Credit Fund separate. They make genuinely

different decisions (short-term revolvers vs. long-term debt).

- Result: 8 agents = 1 environment + 5 firms + 1 equity market + 1 bank

(merged commercial + credit, or keep separate if the LLM calls are cheap).

The 11-agent design is the target architecture. Document it. Build toward it. But do not require all 11 for the first working simulation.

2b. Start Public, Defer Private Mode

The private-start mode with PE/VC terms (liquidation preferences, anti-dilution, board seats, IPO transition) is a research

feature. It doubles the complexity of the capitalization sub-system for a mode most runs won't use.

Recommendation: Implement public_start only for v1. Private mode is Phase 2+. Keep the design in doc 14 but flag it as deferred.

2c. Default All Toggles OFF

The toggle system is well-designed. For v1:

Toggle	v1 Status	Why
entry_exit	ON	Core mechanic, needed for realism
financial_institutions	ON	Core mechanic
ma_enabled	OFF	Complex accounting, defer
leasing_enabled	OFF	Marginal realism gain
stock_comp_enabled	OFF	Marginal realism gain
workforce_detail	OFF	Nice but not load-bearing
working_capital_decisions	OFF	Use defaults (15% AR, 15% AP)
provisions_enabled	OFF	Roll into SGA automatically

This reduces the firm decision JSON from ~20 fields to ~10: price, production, capex, rd_spend, rd_allocation, sga_spend, equity_issuance_request, debt_request, dividends, buybacks.

2d. Compustat: 45 Columns, Not 76

The expanded 76-column panel (doc 12) includes goodwill, leasing, stock comp, restructuring, and other columns that are zero when toggles are OFF. For v1, implement the ~45 core columns that are always populated. Add the remaining 31 columns when the corresponding toggle is implemented.

2e. Environment Agent: Structured + Narrative, Not Pure LLM

The riskiest design decision is having the environment LLM allocate demand. If the LLM produces unreasonable shares (firm with highest price gets most demand), the simulation breaks.

Recommendation: The orchestrator computes demand allocation using the multinomial logit model deterministically. The environment LLM's job is:

- Decide if any events occur (safety scandal, breakthrough, etc.)
- Decide R&D outcomes (stochastic success checks, with orchestrator validation)
- Write the narrative (gazette, dossier updates)
- Apply small adjustments to the deterministic demand (+/-10% per firm, justified)

This gives the environment creative latitude without letting it break the economics. The orchestrator validates that adjustments stay within bounds.

2f. Memory: Start Simple

For v1:

- Within-run memory: Last 4 quarters, full detail. No medium-term or long-term summarization. Just truncate at 4 quarters. If context is too large, truncate to 2 quarters.

- Cross-run memory: Skip entirely for v1. Agents get no historical context from past runs. Add this in Phase 2+.

- Industry Gazette: Generate it (it is valuable context), but do not implement agent-side interpretation or gazette_notes table. Agents read the gazette as part of their quarterly context.

This eliminates the self-summarization LLM calls (saving ~30% of LLM cost per quarter) and the cross-run retrieval

system.

3. Areas Needing More Precision Before Coding

3a. Phase 4 Feasibility Clamping -- Needs Pseudocode

The priority-order clamping is described in prose but the exact algorithm is not shown. Before coding, write pseudocode:

```
def clamp_firm_spending(firm, decisions, available_cash, available_credit):
    available = available_cash + available_credit

    # Priority 1: COGS (mandatory -- can't sell without producing)
    cogs = min(decisions.production * firm.unit_cost, available)
    actual_production = cogs / firm.unit_cost
    available -= cogs

    # Priority 2: Mandatory costs (Phase III, interest, taxes)
    mandatory = firm.phase3_cost + firm.interest_due + firm.taxes_due
    if mandatory > available:
        return DEFAULT # firm cannot meet obligations -> flag for default
    available -= mandatory

    # Priority 3: Discretionary (pro-rata if insufficient)
    requested = decisions.capex + decisions.rd_spend + decisions.sga_spend
    if requested > available:
        scale = available / requested
        actual_capex = decisions.capex * scale
        actual_rd = decisions.rd_spend * scale
        actual_sga = decisions.sga_spend * scale
    else:
        actual_capex, actual_rd, actual_sga = decisions.capex, decisions.rd_spend, decisions.sga_spend
    available -= (actual_capex + actual_rd + actual_sga)

    # Priority 4: Payouts (only from surplus)
    actual_div = min(decisions.dividends, available)
    if firm.retained_earnings < 0:
        actual_div = 0 # cannot pay dividends with negative RE
    available -= actual_div
    actual_buyback = min(decisions.buybacks, available)
    available -= actual_buyback

    return ClampedDecisions(...)
```

3b. Phase 6 Accounting Postings -- Needs Worked Example

Write one complete worked example of a firm quarter:

- Starting balance sheet
- Firm decisions (price=\$90K, production=200, capex=\$15M, R&D=\$25M, SGA=\$10M)
- Market outcomes (units_sold=180, market_share=0.21)
- Full income statement

- Full balance sheet updates
- Full cash flow statement
- All derived values (capability stock, brand stock, unit cost)
- End-of-quarter balance sheet
- Invariant checks (BS identity, cash reconciliation, RE roll-forward)

This worked example is the specification for the accounting module. If the code produces numbers matching the example, the accounting is correct.

3c. Environment Prompt -- Needs a Draft

The most critical prompt in the system is the one sent to the environment agent in Phase 5. Write a complete draft prompt (not a template description, but the actual text that would be sent to an LLM) and include it in the docs. Test it manually with Claude/GPT to see if the output is usable.

3d. Firm Decision Prompt -- Needs a Draft

Same for the firm agent. Write the complete Q2 prompt for a firm that has been operating for one quarter, including:

- Private state
- Public info
- Memory (Q1 only)
- World context summary
- Decision instructions

Test it manually. Check: does the LLM produce valid JSON? Are the numbers reasonable? Does it use the analytical context?

4. Workflow Recommendations

4a. Build Order (Revised)

```
WEEK 1-2: Skeleton
- Config system (YAML loading, toggle management)
- FirmState / QuarterFlows dataclasses
- Accounting postings (from worked example above)
- Invariant validation
-> TEST: hand-craft 3 quarters, verify BS identity, cash reconciliation

WEEK 3-4: Orchestrator Loop
- 9-phase engine with hardcoded inputs
- Feasibility clamping
- Settlement + default check
- Bankruptcy auction + fresh entry
- Multinomial logit demand model (deterministic fallback)
-> TEST: 10 quarters with hardcoded decisions, 1 forced default, 1 entry

WEEK 5-6: Agent Application
- FastAPI server with /turn endpoint
- Mock LLM backend
- Brain: simplified 3-step (analyze -> decide -> store)
- SQLite memory (decisions + observations tables only)
```

- 3-4 core analysis tools (trend, margin, cash_runway, competitor_summary)
- > TEST: POST a context, get valid JSON back, memory.db has entries

WEEK 7-8: Integration

- Orchestrator calls agent servers
- IPO sub-sequence
- Full quarter loop
- Compustat panel writer
- Basic checkpoint/resume
- > TEST: `python -m llm_firm_lab smoke --mock --quarters 10`
All invariants pass. Panel has 50 rows (5 firms * 10 quarters).

WEEK 9-10: LLM Integration + Prompts

- Ollama backend (+ OpenRouter backend)
- Complete prompt templates for all phases
- JSON repair loop
- Gazette generation
- > TEST: 5-quarter run with real LLM. Coherent decisions. Valid JSON.

WEEK 11-12: Full Run + Scoring

- 20-quarter run
- Scoring (equity IRR, pricing errors)
- Environment rating
- Diagnostics
- Basic dashboard (3-4 time-series charts)
- > TEST: Complete run. Firms differentiate. No death spiral. Scores computed.

4b. Integration Testing Strategy

The biggest risk is Phase 4 (integration). The agent-orchestrator handshake must work correctly or nothing works. Recommended approach:

- Mock-to-mock: Orchestrator with hardcoded inputs -> validates accounting
- Mock agents: Orchestrator calls mock agent servers -> validates communication
- One real agent: One firm uses real LLM, rest are mock -> validates prompt/parse
- All real agents, short run: 5 quarters -> validates multi-agent interaction
- Full run: 20+ quarters -> validates stability and scoring

4c. Testing the Environment Agent

The environment agent is the most important and hardest-to-test agent. Strategy:

- Prepare 5 "golden" quarter contexts (different market conditions)
- Run each through the environment prompt 10 times
- Check: are demand allocations within bounds? Are narratives coherent?

Do events have reasonable probabilities? Do R&D outcomes respect thresholds?

- If outputs are erratic: tighten the prompt, add more examples, reduce the environment's degrees of freedom (more deterministic, less creative)

4d. Regression Testing

After each major code change, run:

```
python -m llm_firm_lab smoke --mock --quarters 10 --seed 42
python -m llm_firm_lab diagnostics --run-id smoke_42
```

If invariants fail or diagnostics flag degeneracy, the change broke something. This should take <30 seconds with mock LLMs.

5. Access and Configuration Workflow

5a. User's Pre-Run Checklist

```
1. Choose world:      config/worlds/default/ (or create custom)
2. Choose mode:       public_start (default) or private_start
3. Choose toggles:    Edit config.yaml complexity section
4. Choose agents:     How many firms? Which LLM per agent?
5. Choose regimes:    Information regime + measurement regime
6. Choose seed:       For reproducibility
7. Run:               python -m llm_firm_lab run
8. Inspect:           python -m llm_firm_lab dashboard
```

5b. Per-Agent LLM Configuration

Each agent needs 3 things configured:

- Backend: ollama, openrouter, anthropic, openai, mock
- Model: specific model name for that backend
- API key (if cloud): environment variable name

The config.yaml should support a shorthand for "all agents use the same backend":

```
# Shorthand: all agents use the same LLM
default_llm:
  backend: "openrouter"
  model: "deepseek/deepseek-v3.2"
  api_key_env: "OPENROUTER_API_KEY"

# Override specific agents
agents:
  env_0:
    model: "llama3.2:70b"      # environment gets bigger model
  eqmkt_0:
    backend: "openrouter"
    model: "anthropic/claude-sonnet-4-20250514"
    api_key_env: "OPENROUTER_API_KEY"
```

5c. During-Run Inspection

The user should be able to see what is happening without waiting for the run to finish:

- Terminal output: One summary line per quarter:

```
Q3 2031: Rev=$198M | Firms=5 | Defaults=0 | HHI=2050 | Gen=1,1,1,1,1
Q4 2031: Rev=$225M | Firms=5 | Defaults=0 | HHI=1980 | Gen=1,1,1,1,1
Q1 2032: Rev=$310M | Firms=5 | Defaults=0 | HHI=1920 | Gen=1,1,2,1,1 *** Gen 2!
```

- Gazette: Printed to terminal (abbreviated) and saved to disk

- Dashboard: Live-updating if running --live mode
- Pause: User can press a key to pause after the current quarter completes,

inspect state, and resume

5d. Post-Run Inspection

```
# Summary
python -m llm_firm_lab inspect --run-id run_042

# Specific firm, specific quarter
python -m llm_firm_lab inspect --run firm_0 --quarter 14

# Reasoning trace (why did firm_0 do what it did in Q14?)
python -m llm_firm_lab inspect --run firm_0 --quarter 14 --reasoning

# Gazette for a quarter
python -m llm_firm_lab inspect --quarter 14 --gazette

# Plots
python -m llm_firm_lab plot --type revenue      # all firms revenue
python -m llm_firm_lab plot --type margins     # all firms margins
python -m llm_firm_lab plot --type cash        # all firms cash
python -m llm_firm_lab plot --type rd          # R&D spend + gen advances
python -m llm_firm_lab plot --type equity      # stock prices
python -m llm_firm_lab plot --type bank_capital # institution health

# Dashboard (interactive)
python -m llm_firm_lab dashboard
```

6. Areas Requiring Expansion

6a. Worked Examples

The following worked examples should be written before coding begins:

- One complete quarter of accounting (as described in 3b above)
- One IPO sequence (firm requests \$300M, IBank structures, Equity Market prices at \$15/share, banks offer credit, capital booked)
- One bankruptcy (firm runs out of cash, auction with recovery rates, waterfall pays creditors, slot becomes vacant)
- One entry (slot vacant, environment recommends entry, new firm IPOs with fresh fingerprint)
- One R&D advance (firm crosses Gen 2 threshold, stochastic check succeeds, quality scores jump, manufacturing retooling begins)

These examples serve as integration test specifications. If the code reproduces them exactly, the system is correct.

6b. Prompt Library

Before coding the prompt builder, write complete draft prompts for:

- Firm quarterly decision (the most common prompt, called 5x per quarter)
- Environment market resolution (the most important prompt)

- Equity Market pricing (the most intellectually demanding prompt)
- Firm IPO request (first prompt a firm sees)
- Environment entry decision (when to allow new firms)

Test each manually with a real LLM. Iterate until outputs are consistently valid JSON with reasonable numbers. This is the highest-ROI preparatory work.

6c. Error Catalog

Document the expected failure modes and their resolutions:

Failure	Detection	Resolution
LLM returns invalid JSON	JSON parse fails	Extract first {}, retry 2x, then f
LLM returns absurd price (\$0, \$10B)	Bounds check	Clamp to [min_cogs, 10x_competitor]
LLM returns negative production	Schema validation	Set to 0
LLM ignores budget constraint	Clamping in Phase 4	Automatic, logged
Environment gives 100% share to on	Market share bounds	Re-prompt, then fallback to logit
Environment generates event every	Event frequency check	Re-prompt, limit to world doc rate
Agent server timeout	180s deadline	Use fallback decisions
Agent server crash	Health check fails pre-quarter	Skip agent, use fallback, warn use
Checkpoint file corrupted	Load fails	Roll back to previous checkpoint
Compustat invariant fails	Post-quarter validation	HALT run, dump state for debugging

7. Implementation Notes for Coding

7a. Data Types First

The first code written should be the dataclasses (FirmState, QuarterFlows, MacroState). Every other module depends on these. Use Python dataclasses with `__post_init__` validation. Make them immutable (frozen=True) with explicit `evolve()` methods that produce new instances.

7b. Accounting as Pure Functions

The accounting module should be pure functions:

```
def post_quarter(
    prior_state: FirmState,
    decisions: ClampedDecisions,
    outcomes: MarketOutcomes,
    params: SimParams
) -> tuple[FirmState, QuarterFlows]:
    """Given prior state + decisions + outcomes, produce new state + flows.
    Pure function. No side effects. No database access. No LLM calls."""
```

This makes testing trivial: provide inputs, check outputs.

7c. Agent Server as a Thin Shell

The FastAPI server should do almost nothing:

```
@app.post("/turn")
async def turn(context: TurnContext) -> Decision:
    decision = brain.run_pipeline(context)
    memory.store(context, decision)
```



```
return decision
```

All logic lives in `brain.py` and `analyst.py`, which are independently testable without running a server.

7d. Orchestrator as a Pipeline

The orchestrator's `run_quarter()` should be a linear pipeline:

```
def run_quarter(state: WorldState, agents: dict[str, Agent]) -> WorldState:
    state = phase1_shocks(state)
    state = phase2_ipo(state, agents)
    decisions = phase3_firm_decisions(state, agents)
    decisions = phase4_clamp(state, decisions)
    outcomes = phase5_market_resolution(state, decisions, agents)
    state = phase6_accounting(state, decisions, outcomes)
    terms = phase7_financial_decisions(state, agents)
    state = phase8_settlement(state, terms)
    state = phase9_record_keeping(state)
    validate(state)
    return state
```

Each phase function is independently testable. The state object is the only thing threaded through.

7e. LLM Backend Abstraction

```
class LLMBackend(ABC):
    @abstractmethod
    def complete(self, system: str, user: str, schema: dict | None = None) -> str:
        """Send system + user prompt, return text response."""

class OllamaBackend(LLMBackend): ...
class OpenRouterBackend(LLMBackend): ...
class AnthropicBackend(LLMBackend): ...
class MockBackend(LLMBackend): ...
```

The schema parameter is optional: if provided, the backend can use structured output / tool calling / JSON mode to increase reliability.

7f. File Organization (Pragmatic)

Instead of the sprawling directory tree in doc 05, start flat:

```
src/
  config.py          # YAML loading + validation
  types.py           # All dataclasses
  accounting.py      # Pure functions: post_quarter, validate
  clamping.py        # Feasibility clamping
  demand.py          # Multinomial logit (deterministic fallback)
  orchestrator.py    # run_quarter pipeline
  agent_server.py    # FastAPI server for agents
  brain.py           # Multi-step reasoning pipeline
  analyst.py         # Statistical tools
  memory.py          # SQLite read/write
  llm_backends.py    # All LLM backends in one file
  prompts.py         # All prompt builders in one file
  compustat.py       # CSV writer + validator
```

```

cli.py          # CLI entry point
dashboard.py    # Streamlit app (when needed)
tests/
  test_accounting.py
  test_clamping.py
  test_orchestrator.py
  test_prompts.py
  conftest.py    # Fixtures: sample firms, sample decisions

```

17 files. Refactor into subpackages later when any file exceeds ~500 lines.

8. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Environment LLM produces	High	Critical	Deterministic logit as pr
Firm LLMs produce identic	Medium	High	Fingerprints + randomized
Accounting invariants fai	Medium	High	Pure-function accounting;
Prompts too large for con	Medium	Medium	Token budget enforcement;
Multi-agent coordination	Medium	High	Mock agents first; add on
M&A accounting (goodwill,	High	Medium	Defer to after core works
Private mode (PE/VC terms	High	Medium	Defer; public_start only
Cross-run memory creates	Low	Medium	Defer entirely for v1
Dashboard scope creep del	Medium	Low	Build dashboard LAST; use

9. Definition of "v1 Done"

A working v1 means:

- `python -m llm_firm_lab run --quarters 20 --seed 42` completes without errors
- 5 firms make differentiated decisions (prices, R&D allocations vary)
- At least 1 firm advances to Gen 2 during the run
- At least 1 firm defaults (or comes close) due to cash burn
- Equity prices change quarter to quarter and roughly track firm quality
- Compustat panel passes all hard invariant checks
- Scoring produces equity IRR for each firm
- Environment generates coherent quarterly gazettes
- `python -m llm_firm_lab inspect --run-id run_42` shows meaningful output
- Run completes in under 4 hours (on a single machine with Ollama)